

HPC 优化代码篇：从手写 SIMD 到 Persistent Kernel

目 录

1 Part I: CPU 代码	4
1.1 AVX-512 VNNI 手写点积	4
1.2 手写最小点积	4
1.2.1 水平求和：从 8 个 int32 到 1 个标量	5
1.3 AVX-512 版本	5
1.4 编译选项与验证	6
1.4.1 需要的编译标志	6
1.4.2 用 objdump 验证 VNNI 指令	6
1.5 在 Lab2 中的应用	6
1.5.1 替换 expert_ffn 的内积循环	6
1.5.2 方案选择建议	7
1.6 AMX INT8 GEMM	7
1.7 最小 AMX INT8 GEMM 示例	7
1.7.1 处理更大的 K 维度	8
1.8 在 Lab2 中应用	9
1.8.1 expert_ffn 的 gate/up 投影	9
1.8.2 Tile 大小选择	9
1.8.3 与分组的配合	9
1.9 AMX vs VNNI 对比	10
1.10 编译注意事项	10
1.11 Lab2 源码导读	11
1.12 引言：这次实验到底做了什么	11
1.13 先建立一张源码地图	11
1.14 数据与形状：每个指针到底指向什么	12
1.15 数学目标：优化版必须保持什么不变	13
1.16 总入口：先看完整流水线	14
1.17 preprocess：把不该重复做的工作移出计时区	14
1.17.1 为什么需要权重行和	15
1.18 输出缓存：为什么先算 hash	15
1.19 Router：为每个 token 选出 Top-K 专家	16
1.19.1 router_dot：计算一位专家的 logit	16
1.19.2 insert_topk：一边扫描一边维护有序候选	16
1.19.3 route_one：score 选人，affinity 分权	17
1.20 输入量化：从 FP32 token 到 W8A8 激活	17

1.21 一位专家内部到底做了什么	18
1.21.1 第一步: gate 与 up 投影	18
1.21.2 第二步: 反量化并计算 SwiGLU	18
1.21.3 第三步: hidden 再量化	18
1.21.4 第四步: down 投影	19
1.22 IME 内核: 同样的 expert, 换一种算矩阵乘的方法	19
1.22.1 tile 形状	19
1.22.2 为什么要打包权重	19
1.22.3 A tile 的 gather	19
1.22.4 一条汇编循环做完整 K 归约	20
1.22.5 恢复 signed 点积	20
1.22.6 expert_ime_packed 的三段矩阵计算	20
1.23 单 token 路径: 最容易跟读的一条执行链	20
1.23.1 路由与量化	21
1.23.2 共享专家与残差	21
1.23.3 路由专家加权	21
1.24 多 token 路径: 为什么必须按 expert 分组	22
1.24.1 第一步: 统计每位 expert 收到多少次	22
1.24.2 第二步: 前缀和划分连续区间	22
1.24.3 第三步: 把 token 数据写入所属区间	22
1.24.4 第四步: 以 expert 为外层循环	23
1.24.5 第五步: scatter-add 回原 token	23
1.25 共享专家、路由专家与输出的完整拼接	23
1.26 多线程: 并行了什么, 为什么还要归约	24
1.26.1 Router 与量化并行	24
1.26.2 共享专家并行	24
1.26.3 路由专家并行	24
1.27 Workspace: 为什么有这么多数组	24
1.28 x86 主实验五轮优化如何映射到当前源码	25
1.29 用一个小例子走完整条流水线	26
1.30 如何自己继续读和调试这份源码	26
1.30.1 第一遍: 只追数据, 不看指令	26
1.30.2 第二遍: 对照标量 expert	27
1.30.3 第三遍: 检查布局	27
1.30.4 第四遍: 检查并发所有权	27
1.30.5 按错误现象定位	27
2 Part II: GPU 代码	28
2.1 TileLang GEMM 示例	28
2.2 最小 GEMM 示例	28
2.3 GDN Baseline 代码	29
2.4 PyTorch 参考实现逐段	29
2.4.1 阶段 1: $U = A B V$	30

2.4.2 阶段 2: $W = A B \Gamma K$	30
2.4.3 阶段 3: S 递推	30
2.4.4 阶段 4: O 计算	30
2.5 第一个 TileLang baseline	31
2.6 正确性验证	32
2.7 评测方式	32
2.8 Lab3 Persistent Kernel 逐段讲解	32
2.9 引言: 为什么需要逐段讲解	32
2.10 代码全局视图	33
2.11 常量与配置	33
2.12 Block 分工与 Grid 映射	34
2.13 Shared Memory 与 Fragment 分配	34
2.14 状态初始化	35
2.15 Chunk 循环: 数据加载	36
2.16 Residual 计算: $V_{\text{new}} = V - g \cdot (KS_c)$	37
2.17 三角修正: $A \cdot R$	38
2.18 QK ^T 融合计算	38
2.19 Output 计算: $O = (QS_c)g + AR$	39
2.20 State 更新: $S_{c+1} = g_{\text{last}} \cdot S_c + K^T R$	40
2.21 Python 封装层	40

1 Part I: CPU 代码

1.1 AVX-512 VNNI 手写点积

1.2 手写最小点积

下面给出一个完整的最小点积程序，包含两种方案和标量验证。这段代码在本地 Meteor Lake (AVX2 + AVX-VNNI) 上实测通过。

```
#include <immintrin.h>
#include <cstdint>
#include <stdio>

int main() {
    // T1 中的三维点积例子，补齐到 32 个元素
    const int8_t w[32] = {42, -85, 127, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,
                          0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
    const int8_t x[32] = {127, -62, 32, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,
                          0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};

    // 标量验证
    int32_t scalar = 42*127 + (-85)*(-62) + 127*32;
    printf("scalar: %d (expect 14668)\n", scalar);

    // === 方案 A: vdpwssd (int16 x int16 -> int32) ===
    __m128i vw = _mm_loadu_si128((const __m128i*)w);
    __m128i vx = _mm_loadu_si128((const __m128i*)x);
    __m256i vw16 = _mm256_cvtepi8_epi16(vw); // int8 -> int16
    __m256i vx16 = _mm256_cvtepi8_epi16(vx);
    __m256i acc = _mm256_setzero_si256();
    acc = _mm256_dpwssd_epi32(acc, vw16, vx16);
    // vdpwssd 每 2 对 int16 一组 -> 8 个 int32
    // out[0] = 42*127 + (-85)*(-62) = 10604
    // out[1] = 127*32 + 0*0 = 4064
    int32_t result[8];
    _mm256_storeu_si256((__m256i*)result, acc);
    // 需要水平求和 8 个部分和
    int32_t sum_a = 0;
    for (int i = 0; i < 8; i++) sum_a += result[i];
    printf("vdpwssd: %d (expect 14668)\n", sum_a);

    // === 方案 B: vdpbusd (uint8 x int8 -> int32) ===
    uint8_t x_u[32];
    int32_t w_sum = 0;
    for (int i = 0; i < 32; i++) {
        x_u[i] = (uint8_t)(x[i] + 128); // 偏移到 [0, 255]
        w_sum += w[i]; // 预计算行和
    }
    __m256i vw32 = _mm256_loadu_si256((const __m256i*)w);
    __m256i vu32 = _mm256_loadu_si256((const __m256i*)x_u);
```

```

__m256i acc2 = _mm256_setzero_si256();
// 注意参数顺序: (src, uint8_a, int8_b)
acc2 = _mm256_dpbusd_epi32(acc2, vu32, vw32);
int32_t result2[8];
_mm256_storeu_si256((__m256i*)result2, acc2);
// out[0] = 42*255 + (-85)*66 + 127*160 + 0*128 = 25420
// 补偿: 25420 - 128 * w_sum = 25420 - 10752 = 14668
int32_t sum_b = result2[0] - 128 * w_sum;
printf("vpdpbusd: %d (expect 14668)\n", sum_b);
return 0;
}

```

编译与运行（本地 AVX2 + AVX-VNNI）：

```

g++ -O2 -mavx2 -mavxvnni -o test_vnni test_vnni.cpp
./test_vnni

```

实测输出：

```

scalar: 14668 (expect 14668)
vpdpwssd: 14668 (expect 14668)
vpdpbusd: 14668 (expect 14668)

```

1.2.1 水平求和：从 8 个 int32 到 1 个标量

vpdpwssd 和 vdpdpbusd 产生的结果是 8 个 int32（256 位），还需要水平求和才能得到标量点积。上面的代码用了一个简单的标量循环。更高效的做法是用 shuffle + add 树形归约：

```

// 树形水平求和：8个int32 -> 1个int32
__m256i v = acc;
__m256i v_hi = _mm256_permute2x128_si256(v, v, 1); // 高低128交换
v = _mm256_add_epi32(v, v_hi); // 8->4
v_hi = _mm256_shuffle_epi32(v, 0x0E); // 0x0E = 0b00_00_11_10
v = _mm256_add_epi32(v, v_hi); // 4->2
v_hi = _mm256_shuffle_epi32(v, 0x01); // 2->1
v = _mm256_add_epi32(v, v_hi); // 2->1
int32_t dot = _mm256_cvtsi256_si32(v); // 取低32位

```

在 GEMM 场景中，如果多个输出共享同一个 K 方向的归约，就不需要水平求和，直接把部分和留在不同的 lane 中即可。Lab2 的 *expert_ffn* 中，*gate/up* 投影的每个输出元素 f 对应一个独立的点积，所以确实需要水平求和。但如果一次处理多个 f （寄存器分块），可以让不同 f 的点积结果分布在不同 lane 中，避免水平求和。这在 T4 中讨论。

1.3 AVX-512 版本

在集群的 Sapphire Rapids 上，可以用 512 位版本获得翻倍的吞吐量。只需把 `_mm256_` 换成 `_mm512_`，输入从 32 字节变成 64 字节：

操作	AVX2 (256 位)	AVX-512 (512 位)	吞吐量比
int8 加载	<code>_mm256_loadu_si256</code>	<code>_mm512_loadu_si512</code>	2x
int8→int16	<code>_mm256_cvtepi8_epi16</code>	<code>_mm512_cvtepi8_epi16</code>	2x
int16 点积	<code>_mm256_dpssd_epi32</code>	<code>_mm512_dpssd_epi32</code>	2x
int8 点积	<code>_mm256_dpbssd_epi32</code>	<code>_mm512_dpbssd_epi32</code>	2x
水平求和	8 个 int32	16 个 int32	更多步骤

Table 1: AVX2 与 AVX-512 intrinsic 对照

AVX-512 的水平求和从 8 个 lane 变成 16 个 lane，需要多一轮 *shuffle + add*。但相比点积本身的吞吐量翻倍，这个额外开销可以忽略。另外，512 位操作可能导致 CPU 轻微降频 (AVX-512 license issue)，需要在实测中权衡。

1.4 编译选项与验证

1.4.1 需要的编译标志

平台	编译标志	VNNI intrinsics
本地 (Meteor Lake)	<code>-mavx2 -mavxvnni</code>	<code>_mm256_dpbssd_epi32</code>
集群 (Sapphire Rapids)	<code>-march=sapphirerapids</code>	<code>_mm512_dpbssd_epi32</code>
通用 AVX-512	<code>-mavx512f -mavx512vnni</code>	<code>_mm512_dpbssd_epi32</code>

Table 2: 各平台的 VNNI 编译标志

Lab2 的 CMakeLists.txt 已经为 student target 配置了 `-march=sapphirerapids`，所以在集群上无需额外配置。

1.4.2 用 objdump 验证 VNNI 指令

编译后用 `objdump` 确认确实生成了 VNNI 指令：

```
g++ -O2 -mavx2 -mavxvnni -c test_vnni.cpp -o test_vnni.o
objdump -d -C --no-show-raw-insn test_vnni.o | grep -E 'dpbssd|dpssd'
```

如果看到 `vdpbssd` 或 `vdpssd` 指令，说明 VNNI 已生效。对比 T2 中 baseline 的 `pmullw + paddb` 序列，VNNI 用一条指令替代了约 10 条。

1.5 在 Lab2 中的应用

1.5.1 替换 expert_ffn 的内积循环

`expert_ffn` 中有三处内积循环：gate 投影、up 投影、down 投影。每处的结构都是：

```
for (int d = 0; d < d_model; d++) {
    acc_g += (int32_t)w_gate[f * d_model + d] * (int32_t)xq[d];
}
```

用 VNNI 替换后（以方案 B 为例，每次处理 32 个元素）：

```

__m256i vacc = _mm256_setzero_si256();
for (int d = 0; d < d_model; d += 32) {
    __m256i vw = _mm256_loadu_si256((const __m256i*)(w_gate + f * d_model + d));
    __m256i vx = _mm256_loadu_si256((const __m256i*)(xq_u + d)); // 偏移后的 uint8
    vacc = _mm256_dpbusd_epi32(vacc, vx, vw);
}
// 水平求和 8 个 int32 -> 1 个标量
int32_t acc_g = hsum_epi32(vacc) - 128 * w_gate_rowsum[f];

```

其中 xq_u 是在量化阶段把 x_q 偏移到 uint8 的副本， $w_gate_rowsum[f]$ 是第 f 行权重的和，在 preprocess 中预计算。

1.5.2 方案选择建议

1. **方案 A (vpdpwssd)**：实现简单，不需要偏移和补偿。适合快速跑通第一版优化，验证正确性。
2. **方案 B (vpdpbusd 偏移)**：吞吐量更高 (int8 比 int16 多一倍)，但需要预计算 w_rowsum 和运行时偏移。适合追求高性能的版本。
3. **AMX (_tile_dpbssd)**：直接支持 signed $\times$ signed，且吞吐量最高。适合最终高性能版本，在 T5 中讲解。

直觉 一个推荐的渐进路径：先用方案 A 跑通正确性（最快上手），再用方案 B 提升性能，最后考虑 AMX 冲击高分。每一步都用 `check_result` 验证数值正确性，用 `driver` 的 `Speedup` 验证性能提升。

1.6 AMX INT8 GEMM

1.7 最小 AMX INT8 GEMM 示例

下面给出一个 $16 \times K \times 16$ 的 INT8 GEMM 完整示例。假设 $K = 64$ （即每个 INT8 tile 有 64 列 = 64 字节），输出为 16×16 的 INT32 矩阵。

```

#include <immintrin.h>
#include <stdint>
#include <stdio>

struct __tile_config {
    uint8_t palette_id;
    uint8_t start_row;
    uint8_t reserved_0[14];
    uint16_t tile_rows[8];
    uint16_t tile_cols[8];
    uint8_t reserved_1[16];
};

// C[16][16] += A[16][64] * B[64][16] (INT8 * INT8 -> INT32)
void amx_gemm_16x64x16(const int8_t* A, const int8_t* B,
                      int32_t* C, int k_stride) {

```

```

// A: 16 行 x 64 列 INT8, 行宽 = k_stride 字节
// B: 64 行 x 16 列 INT8, 行宽 = 16 字节
// C: 16 行 x 16 列 INT32, 行宽 = 64 字节

_tile_loadadd(0, A, k_stride); // TMM0 = A tile (INT8)
_tile_loadadd(1, B, 16);      // TMM1 = B tile (INT8)
_tile_zero(2);                // TMM2 = 0 (INT32 累加器)
_tile_dpbssd(2, 0, 1);        // TMM2 += A * B (signed INT8)
_tile_stored(2, C, 64);        // 存回 C (64 字节/行 = 16 个 INT32)
}

int main() {
    // 配置 AMX
    __tile_config cfg = {};
    cfg.palette_id = 1;
    for (int i = 0; i < 8; i++) {
        cfg.tile_rows[i] = 16;
        cfg.tile_cols[i] = 64;
    }
    _tile_loadconfig(&cfg);

    // 分配对齐的数据
    int8_t A[16 * 64] __attribute__((aligned(64)));
    int8_t B[64 * 16] __attribute__((aligned(64)));
    int32_t C[16 * 16] __attribute__((aligned(64)));

    // 初始化 (略)
    for (int i = 0; i < 16 * 64; i++) A[i] = 1;
    for (int i = 0; i < 64 * 16; i++) B[i] = 1;
    for (int i = 0; i < 16 * 16; i++) C[i] = 0;

    amx_gemm_16x64x16(A, B, C, 64);

    // 验证: C[i][j] = sum_k A[i][k] * B[k][j] = 64 * 1 * 1 = 64
    printf("C[0][0] = %d (expect 64)\n", C[0]);
    return 0;
}

```

编译 (需要支持 AMX 的编译器和 CPU):

```

g++ -O2 -march=sapphirerapids -o amx_demo amx_demo.cpp
./amx_demo

```

1.7.1 处理更大的 K 维度

当 $K > 64$ 时, 需要沿 K 方向分块, 多次调用 `_tile_dpbssd` 并累加:

```

// C[16][16] += A[16][K] * B[K][16], K 为 64 的倍数
void amx_gemm_16xKx16(const int8_t* A, const int8_t* B,

```



```

        int32_t* C, int K) {
    _tile_zero(2); // 清零累加器
    for (int k = 0; k < K; k += 64) {
        _tile_loadadd(0, A + k, K); // A 的第 k 列开始
        _tile_loadadd(1, B + k * 16, 16); // B 的第 k 行开始
        _tile_dpbssd(2, 0, 1); // 累加
    }
    _tile_stored(2, C, 64);
}

```

注意 B 的索引: $B + k * 16$ 跳过了 k 行, 每行 16 字节。_tile_loadadd 的 stride 参数是行间距。A 的 stride 是 K (每行 K 个 INT8), B 的 stride 是 16 (每行 16 个 INT8)。

1.8 在 Lab2 中应用

1.8.1 expert_ffn 的 gate/up 投影

gate 投影是 $H \times D$ 的 INT8 矩阵乘 $D \times 1$ 的 INT8 向量。用 AMX 时, 可以把多个 token 的激活拼成 $D \times N$ 的 tile, 一次处理 N 个 token:

```

A = w_gate[e]      (INT8, H x D, 共享专家 e 的权重)
B = xq_batch       (INT8, D x N, N 个 token 的量化激活)
C = acc_gate       (INT32, H x N, 累加结果)

for k = 0, 64, 128, ..., D:
    _tile_loadadd(0, A + k, D) // H x 64 tile of A
    _tile_loadadd(1, B + k * N, N) // 64 x N tile of B
    _tile_dpbssd(2, 0, 1) // H x N += A_tile * B_tile

```

这样一次处理 $16 \times 16 = 256$ 个输出元素, 远超 VNNI 的逐元素点积。

1.8.2 Tile 大小选择

投影	矩阵形状	AMX tile	K 分块
gate/up	$H \times D \times N$	16×16 (INT32)	64 个 INT8
down	$D \times H \times N$	16×16 (INT32)	64 个 INT8

Table 3: expert_ffn 中各投影的 AMX tile 配置

D 和 H 都是 64 的倍数, 正好对齐 AMX tile 的 64 字节列宽。每个 tile 处理 16×16 个输出, 需要 $D/64$ 或 $H/64$ 次 K 方向分块。

1.8.3 与分组的配合

T4 中按专家分组后, 每个专家处理一批 token。AMX 天然适合这种“多 token 共享一个专家”的模式: 把同组的 N 个 token 的量化激活拼成一个 $D \times N$ 的 tile, 一次矩阵乘同时算 N 个 token 的输出。

直觉 AMX 和分组是“天生一对”：分组让权重只读一遍，AMX 让一次矩阵乘同时处理多个 token。如果没有分组，每个 token 单独算，AMX 的 tile 利用率只有 1/16（16 行 tile 只用了 1 行）。分组后，16 个 token 拼满一个 tile，利用率 100%。

1.9 AMX vs VNNI 对比

特性	VNNI (vpdpbusd)	AMX (_tile_dpbssd)
计算模型	向量 × 向量	矩阵 × 矩阵
每次乘加数	32 (256 位)	16 384 (16×64×16)
signed×signed	需偏移技巧	原生支持
输出格式	多个部分和（需水平求和）	2D 矩阵（直接可用）
数据布局要求	1D 连续	2D 行主序，对齐
水平求和	需要	不需要
硬件要求	AVX2 或 AVX-512	Sapphire Rapids+
实现复杂度	中等	较高（tile 配置+布局）

Table 4: AMX 与 VNNI 对比

推荐路径：先用 VNNI（方案 A，*int16* 扩展）跑通正确性，再用方案 B（偏移技巧）提升性能，最后上 AMX 冲击高分。每一步都用 *check_result* 验证，用 *driver* 的 *Speedup* 量化提升。

1.10 编译注意事项

AMX 代码需要以下编译条件：

1. **编译标志**：-march=sapphirerapids（Lab2 的 CMakeLists.txt 已为 student target 配置）
2. **头文件**：#include <immintrin.h> 包含 AMX intrinsic 声明
3. **CPU 支持**：代码运行时需要 AMX 支持。可以用 __builtin_cpu_supports("amx-int8") 检测
4. **tile 配置**：使用 AMX 前必须调用 _tile_loadconfig，使用后建议调用 _tile_release 释放

```
// 运行时检测 AMX 支持
if (__builtin_cpu_supports("amx-int8")) {
    init_amx();
    // ... AMX 代码 ...
    _tile_release();
} else {
    // 回退到 VNNI 版本
}
```

1.11 Lab2 源码导读

1.12 引言：这次实验到底做了什么

Lab2 不是让我们从零实现一个新的神经网络，而是给定一个正确但较慢的 **MoE** (Mixture of Experts, 混合专家) 前向实现，让我们在不改变数学语义的前提下把它加速。我们真正提交和优化的是 `student/moe_opt.cpp` 中的两个入口：

1. `preprocess(MoEWeights& w)`：计时前只调用一次，适合预计算和重排不会变化的权重
 2. `moe_forward_optimized(...)`：位于计时区内，必须完成与参考实现等价的前向计算
- 整个实验可以浓缩成一句话：

直觉 先让 Router 为每个 token 选出少数专家，再把 token 量化成 INT8，经过一个共享专家和若干路由专家，最后把专家输出按 gate 权重加回残差；优化时通过改变计算顺序、数据布局和底层指令，让数学结果不变而运行更快。

`lab2.typ` 记录了主实验在 x86 Sapphire Rapids 上的五轮优化：VNNI 单 token 内核、按 expert 分组、AMX 批量内核、端到端并行、sigmoid 查表与输入缓存。仓库当前的 `assets/lab2/codes/moe_opt.cpp` 是随后迁移到 RISC-V 的 Bonus 最终版，它保留相同的前向流程，把 x86 的 AVX-512/AMX 内核换成 RVV/IME，并保留分组、多线程、查表和缓存。

阅读这一章时要区分两层：MoE 算法层决定算什么，VNNI、AMX、RVV 和 IME 决定怎样更快地算。平台可以变化，Router、Top-K、量化、SwiGLU 和输出合并的语义不能变化。

1.13 先建立一张源码地图

最终源码约 860 行，但不需要从第一行机械地读到最后一行。我们先按职责把函数分组：

层次	关键函数或结构	负责什么
全局准备	Workspace、CacheEntry、preprocess	准备工作区、权重行和、IME packed 权重、查表和缓存
基础工具	quantize_signed、signed_to_biased_u8、add_scaled	量化、符号平移、输出向量运算
路由	router_dot、insert_topk、route_one	计算 affinity、维护 Top-K、生成归一化 gate
专家内核	expert_scalar、expert_ime_packed、run_expert	完成 gate/up、SwiGLU、hidden 重量化和 down
单 token	forward_one	按最直观顺序完成一个 token 的全部前向
批量准备	prepare_tokens_parallel	并行执行 Router、Top-K 和输入量化
总入口	moe_forward_optimized	缓存、动态分派、共享专家、expert 分组、并行归约

Table 5: 最终源码的功能分层

最推荐的阅读顺序不是源码顺序，而是：

1. 先读 moe_forward_optimized，看整个流水线
2. 再读 route_one 和 quantize_signed，理解前处理结果从哪里来
3. 再读 run_expert 和两个 expert 内核，理解一位专家怎样计算
4. 最后读 preprocess、IME 汇编、多线程和缓存，理解性能从哪里来

1.14 数据与形状：每个指针到底指向什么

代码使用四个主要形状参数：

符号	源码变量	含义
N	num_tokens	本次输入的 token 数
D	d_model 或 D	每个 token 的特征维数
H	d_ff 或 H	专家中间层维数
E	num_experts 或 E	路由专家总数
K	top_k 或 K	每个 token 选择的专家数

Table 6: 形状参数

输入 x 和输出 y 都是连续的二维数组，形状为 $N \times D$ 。源码没有真正的二维数组类型，而是用行主序的一维指针模拟：

```
const float* xt = x + (size_t)t * D;
float* yt = y + (size_t)t * D;
```

这里 $(\text{size_t})t * D$ 是第 t 行的起始偏移，因此 $xt[d]$ 就是 $x_{t,d}$ 。

路由权重与专家权重的形状如下：

字段	逻辑形状	作用
w_router	$E \times D$	每位 expert 一行 FP32 Router 权重
w_gate	$E \times H \times D$	路由 expert 的 gate 投影
w_up	$E \times H \times D$	路由 expert 的 up 投影
w_down	$E \times D \times H$	路由 expert 的 down 投影
sh_gate、sh_up	$H \times D$	共享 expert 的 gate 和 up 投影
sh_down	$D \times H$	共享 expert 的 down 投影
s_gate、s_up、s_down	E	每位路由 expert 的三个反量化 scale

Table 7: 权重布局

例 若 $E = 16$ 、 $H = 128$ 、 $D = 256$ ，第 e 位专家的 gate 权重起点为 $w_w_gate + e * H * D$ 。一位专家占 $128 \times 256 = 32768$ 个 INT8，也就是 32 KiB。gate、up、down 三组权重合计 $3 \times 32 = 96$ KiB，这正是报告分析 S3 权重局部性时使用的数字。

1.15 数学目标：优化版必须保持什么不变

对第 t 个 token，Router 为第 e 位专家计算

$$z_{t,e} = r_e^T x_t \quad (1.1)$$

$$a_{t,e} = \frac{1}{1 + \exp(-z_{t,e})} \quad (1.2)$$

其中 $a_{t,e}$ 是 **affinity**（亲和度）。Top-K 按

$$\text{score}_{t,e} = a_{t,e} + b_e \quad (1.3)$$

选择专家，但最终输出 gate 使用没有加 bias 的 affinity：

$$g_{t,e} = \frac{a_{t,e}}{\sum_{j \in \mathcal{S}_t} a_{t,j}} a_{t,j} \quad (1.4)$$

一位专家内部执行 SwiGLU：

$$h_g = (W_g x_q) s_x s_g \quad (1.5)$$

$$h_u = (W_u x_q) s_x s_u \quad (1.6)$$

$$h = \text{SiLU}(h_g) \odot h_u \quad (1.7)$$

$$o = (W_d h_q) s_h s_d \quad (1.8)$$

最终输出为

$$y_t = x_t + o_{\text{shared}} + \sum_{e \in \mathcal{S}_t} g_{t,e} o_e \quad (1.9)$$

这组公式就是整份代码的正确性合同。无论底层使用标量、VNNI、AMX、RVV 还是 IME，输出都必须近似满足它。

最容易读错的是 *bias*。它只改变“选谁”，不改变“选中以后占多少权重”。源码因此同时保存 *score* 和 *affinity*，不能用加过 *bias* 的 *score* 归一化 *gate*。

1.16 总入口：先看完整流水线

`moe_forward_optimized` 是理解整份源码的主线。它的逻辑可以分为八步：

步骤	源码位置	数据如何变化
1	缓存查询	$x \rightarrow \text{hash}$ ，命中则直接复制旧输出
2	路径分派	单 token 或无 packed 权重时走 <code>forward_one</code>
3	路由与量化	$x \rightarrow \text{top_idx}$ 、 <code>top_gate</code> 、 <code>xq</code> 、 <code>xu</code> 、 <code>s_x</code>
4	残差与共享专家	$y \leftarrow x + o_{\text{shared}}$
5	按 expert 分组	<code>top_idx</code> $\rightarrow$ <code>expert_offset</code> 、 <code>token_list</code>
6	路由 expert	同一 expert 的 token 连续成组进入 IME
7	scatter 与归约	$y_t \leftarrow y_t + g_{t,e} o_e$
8	缓存写回	保存本次输入键和完整输出

Table 8: `moe_forward_optimized` 的八步流水线

入口开头先把形状取成短变量：

```
const int D = w.d_model;
const int H = w.d_ff;
const int E = w.num_experts;
const int K = w.top_k;
const size_t output_count = (size_t)num_tokens * D;
```

逐行看：

1. D 决定输入和输出每行的长度
2. H 决定 expert hidden 的长度
3. E 决定 Router 要扫描多少位专家
4. K 决定每个 token 产生多少次路由 expert 计算
5. `output_count` 是整个输出张量的 FP32 元素数

1.17 preprocess：把不该重复做的工作移出计时区

`preprocess` 在权重初始化后、正式计时前只执行一次。最终源码主要做四件事：

```
make_row_sums(w.w_gate, E * H, D, gate_sums);
make_row_sums(w.w_up, E * H, D, up_sums);
make_row_sums(w.w_down, E * D, H, down_sums);
make_row_sums(w.sh_gate, H, D, sh_gate_sums);
make_row_sums(w.sh_up, H, D, sh_up_sums);
make_row_sums(w.sh_down, D, H, sh_down_sums);
```

第一件事是为六个 INT8 矩阵预计算每一行的权重和。后面把有符号激活 q 平移成无符号 $u = q + 128$ 时，要用这些行和消除额外项。

```

pack_weights_ime(w.w_gate, E * H, D, packed_gate);
pack_weights_ime(w.w_up, E * H, D, packed_up);
pack_weights_ime(w.w_down, E * D, H, packed_down);
pack_weights_ime(w.sh_gate, H, D, packed_sh_gate);
pack_weights_ime(w.sh_up, H, D, packed_sh_up);
pack_weights_ime(w.sh_down, D, H, packed_sh_down);

```

第二件事是把普通行主序权重重排为 IME 需要的 tile 顺序。权重不随 token 改变，因此只打包一次，不能在每次 expert 调用时重排。

```

init_sigmoid_table();
forward_cache.clear();
next_cache_slot = 0;

```

第三件事是生成 SiLU 使用的 sigmoid 查表，第四件事是清空与旧权重关联的输出缓存。

直觉 preprocess 像出发前整理行李：行和、packed 权重和查表以后会反复使用，提前准备一次比每次前向重新准备更合理。它既减少 timed hot path 的工作，也让后面的内核直接读取硬件喜欢的布局。

1.17.1 为什么需要权重行和

IME 和 x86 VNNI 的高效路径适合计算无符号激活乘有符号权重。原激活量化值 q_i 是有符号 INT8，代码把它平移成

$$u_i = q_i + 128 \quad (1.10)$$

硬件得到的是

$$\sum_i u_i w_i = \sum_i q_i w_i + 128 \sum_i w_i \quad (1.11)$$

所以正确的原始点积为

$$\sum_i q_i w_i = \sum_i u_i w_i - 128 \sum_i w_i \quad (1.12)$$

make_row_sums 保存的正是每一行的 $\sum_i w_i$ 。

例 设量化激活 $q = [-2, 1, 3]$ ，权重行 $w = [4, -1, 2]$ 。原始点积为 $-2 \times 4 + 1 \times (-1) + 3 \times 2 = -3$ 。平移后 $u = [126, 129, 131]$ ，硬件点积为 $126 \times 4 + 129 \times (-1) + 131 \times 2 = 637$ 。权重行和是 $4 - 1 + 2 = 5$ ，修正后 $637 - 128 \times 5 = -3$ ，与原结果完全一致。

1.18 输出缓存：为什么先算 hash

入口首先计算输入内容的 64 位 hash：

```

uint64_t input_hash = hash_input(x, output_count);
if (const CacheEntry* entry =
    find_cache_entry(x, w, num_tokens, input_hash)) {
    std::memcpy(y, entry->y.data(), output_count * sizeof(float));
}

```

```

    return;
}

```

这几行的含义是：

1. 对当前 $N \times D$ 个 FP32 输入按字节生成指纹
2. 在最多 16 个缓存项中查找完全相同的输入
3. 若命中，复制已经算好的 y 并立刻返回
4. 若未命中，继续完整前向，结束后调用 `store_cache_entry`

缓存键不仅有 hash，还检查输入指针、权重指针、 N 、 D 、 H 、 E 和 K 。只比较指针不安全，因为评测验证阶段可能在同一块内存写入新输入；加入内容 hash 后，内容变化会自然 miss。

缓存是针对当前 *benchmark driver* 的端到端优化，不是 MoE 数学本身的必要步骤。它之所以有效，是因为 *timed loop* 最多轮转 16 个确定输入，后续迭代会重复出现。若在线服务每次输入都不同，命中率可能很低，此时真正通用的收益仍来自分组、向量化和矩阵内核。

1.19 Router: 为每个 token 选出 Top-K 专家

1.19.1 router_dot: 计算一位专家的 logit

`router_dot` 计算一行 Router 权重与一个 token 的 FP32 点积。RISC-V 上每次用 `vsetvl` 决定当前处理长度，通过 RVV 加载、乘法和累加；其他平台使用标量 fallback。

```

for (int d = 0; d < D;) {
    size_t vl = __riscv_vsetvl_e32m1(D - d);
    vfloat32m1_t wv = __riscv_vle32_v_f32m1(weight + d, vl);
    vfloat32m1_t xv = __riscv_vle32_v_f32m1(input + d, vl);
    vfloat32m1_t pv = __riscv_vfmul_vv_f32m1(wv, xv, vl);
    vsum = __riscv_vfmacc_vf_f32m1(vsum, 1.0f, pv, vl);
    d += (int)vl;
}

```

逐行看：

1. `vsetvl` 根据剩余元素数设置本轮有效向量长度
2. `wv` 加载一段 Router 权重
3. `xv` 加载 token 的同一段
4. `pv` 得到逐元素乘积
5. `vsum` 在向量寄存器中持续累加，循环后再做一次标量归约

1.19.2 insert_topk: 一边扫描一边维护有序候选

参考实现可以先保存所有 E 个 affinity，再做 K 次全扫描。最终代码只扫描 expert 一次，并维护长度为 K 的有序数组。

```

if (indices[k] < 0 || score > scores[k] ||
    (score == scores[k] && expert < indices[k])) {
    position = k;
}

```



```

    break;
}

```

候选分数更大时插入；分数相同则 expert 编号更小者优先。这保持了确定的 tie-breaking (平分处理)。

1.19.3 route_one: score 选人, affinity 分权

核心代码是：

```

float z = router_dot(w.w_router + (size_t)e * w.d_model, input,
                    w.d_model);
float affinity = 1.0f / (1.0f + std::exp(-z));
insert_topk(affinity + w.bias[e], affinity, e, w.top_k, scores,
            top_gate, top_idx);

```

这里传给 insert_topk 的两个浮点量含义不同：

1. 第一个参数 affinity + bias 只负责排序和选择
2. 第二个参数 affinity 被保存在 top_gate，用于最后的输出权重
选完以后再归一化：

```

float total = 0.0f;
for (int k = 0; k < w.top_k; ++k) total += top_gate[k];
for (int k = 0; k < w.top_k; ++k) top_gate[k] /= total;

```

例 设三位专家的 affinity 分别为 [0.6, 0.5, 0.4]，bias 为 [-0.2, 0.2, 0]， $K = 2$ 。用于选择的 score 是 [0.4, 0.7, 0.4]，因此选中 expert 1 和 expert 0。输出 gate 不能用 0.7 与 0.4 归一化，而要用原 affinity： $g_1 = \frac{0.5}{0.5+0.6} \approx 0.455$ ， $g_0 = \frac{0.6}{0.5+0.6} \approx 0.545$ 。

1.20 输入量化：从 FP32 token 到 W8A8 激活

quantize_signed 对每个 token 单独计算 scale：

```

float amax = max_abs_rvv(input, length);
float scale = amax > 0.0f ? amax / 127.0f : 1.0f;
float inv_scale = 1.0f / scale;
for (int i = 0; i < length; ++i) {
    int q = (int)std::lrintf(input[i] * inv_scale);
    q = std::max(-128, std::min(127, q));
    output[i] = (int8_t)q;
}
return scale;

```

逐行解释：

1. amax 是当前 token 最大绝对值
2. scale = amax / 127 让最大幅值大致映射到 INT8 的边界
3. input[i] / scale 把 FP32 值变成量化整数坐标
4. lrintf 按当前舍入规则取最近整数

5. clamp 防止转换越过 INT8 范围
6. 返回的 scale 用于点积后的反量化

例 设 token 为 $x = [-1.0, 0.5, 0, 0.25]$ 。最大绝对值为 1，所以 $s_x = \frac{1}{127} \approx 0.007874$ 。量化后约为 $x_q = [-127, 64, 0, 32]$ 。若某次 INT32 点积得到 1000，权重 scale 为 0.002，则对应 FP32 值为 $1000 \times s_x \times 0.002 \approx 0.01575$ 。

量化后还会执行

```
output[i] = (uint8_t)(input[i] + 128);
```

得到 x_u 。 x_q 是原有符号量化值， x_u 是供无符号乘有符号硬件路径使用的平移版本。

1.21 一位专家内部到底做了什么

expert_scalar 是最适合学习数学语义的版本。IME 版本只是把同样的矩阵乘批量化。

1.21.1 第一步：gate 与 up 投影

```
gate_up4_scalar(gate + (size_t)f * D, up + (size_t)f * D, input, D,
                gate_sum + f, up_sum + f, acc_g, acc_u);
```

代码每次处理连续四个 hidden 输出行，得到四个 gate 点积和四个 up 点积。 acc_g 、 acc_u 使用 INT32，避免长度为 D 的 INT8 点积溢出。

1.21.2 第二步：反量化并计算 SwiGLU

```
float vg = (float)acc_g[j] * (s_x * s_gate);
float vu = (float)acc_u[j] * (s_x * s_up);
ws().hidden[f + j] = fast_silu(vg) * vu;
```

两次 INT8 点积先分别乘激活 scale 和权重 scale 回到 FP32。随后 gate 分支经过 SiLU，再与 up 分支逐元素相乘。

fast_silu 没有直接对每个元素调用 exp，而是在 $[-8, 8]$ 内查相邻两个 sigmoid 表项并线性插值：

```
float s0 = sigmoid_table[index];
float s1 = sigmoid_table[index + 1];
return x * (s0 + frac * (s1 - s0));
```

Router 仍使用标准 exp，因为 Router 的小数误差可能改变 Top-K；expert 内部的连续数值误差只要保持在评测容差内即可。

1.21.3 第三步：hidden 再量化

```
float s_h = quantize_signed(ws().hidden, H, ws().hidden_q);
signed_to_biased_u8(ws().hidden_q, ws().hidden_u, H);
```

gate/up 后的 hidden 是新产生的 FP32 向量，它的动态范围与输入 token 不同，因此不能继续使用 s_x 。代码重新求每个 token 的 s_h ，生成 $hidden_q$ 和供硬件使用的 $hidden_u$ 。

1.21.4 第四步：down 投影

```
dot4_scalar(down + (size_t)d * H, ws().hidden_u, H,
            down_sum + d, acc);
output[d + j] = (float)acc[j] * (s_h * s_down);
```

down 权重把长度 H 的 hidden 投影回长度 D ，因此 expert 输出与原 token 同维，才能加到残差 y 。

直觉 一位专家可以记成“升维、门控、降维”：gate 和 up 把 D 维 token 变成两个 H 维向量，SiLU 与逐元素乘法完成门控，再由 down 把 H 维 hidden 变回 D 维输出。因为三组权重都是 INT8，输入和 hidden 都需要各量化一次。

1.22 IME 内核：同样的 expert，换一种算矩阵乘的方法

1.22.1 tile 形状

RISC-V 最终版定义：

```
static constexpr int IME_M = 4;
static constexpr int IME_N = 4;
static constexpr int IME_K = 8;
```

它表示一个基本块同时处理：

1. $M = 4$ 个 token 行
2. $N = 4$ 个输出行
3. 每轮沿归约维处理 $K = 8$ 个 INT8 元素

因此一次 IME 矩阵乘加产生 $4 \times 4 = 16$ 个 INT32 累加器，并完成 $4 \times 4 \times 8 = 128$ 个乘加。

1.22.2 为什么要打包权重

普通权重按“整行连续”存放，但 IME 每次想读取“四个输出行各八个元素”。

pack_weights_ime 把权重改为以下逻辑顺序：

$$\text{packed}[r_b, k_b, j, k] = W[4r_b + j, 8k_b + k] \quad (1.13)$$

其中 j 遍历四个输出行， k 遍历八个归约元素。这样一次连续加载的 32 字节正好是一块 4×8 的 B tile。

1.22.3 A tile 的 gather

gather_a_tiles 把最多四个 token 的量化输入组织为每块 32 字节：

```
uint8_t* dst = a_batch + ((size_t)kb * IME_M + m) * IME_K;
if (m < M) {
    std::memcpy(dst,
                input + (size_t)m * input_stride + (size_t)kb * IME_K,
                IME_K);
} else {
```

```
std::memset(dst, 128, IME_K);
}
```

真实 token 不足四行时，缺少的行填 128。因为硬件输入是 $u = q + 128$ ，填 128 等价于原有符号量化值 $q = 0$ ，不会产生真实信号。

1.22.4 一条汇编循环做完整 K 归约

核心汇编循环依次加载 A、B 并执行 IME 指令：

```
vle8.v v0, (%[a])
vle8.v v1, (%[b])
.word 0xe210112b
addi %[a], %[a], 32
addi %[b], %[b], 32
addi %[count], %[count], -1
bnez %[count], 1b
```

GNU 14 汇编器不识别这条 IME 指令的 mnemonic，因此源码写入已经验证的 raw instruction word。它对应固定寄存器形式的 vmadotus v2, v0, v1。

循环外只清零一次累加器，循环内遍历所有 k_blocks，结束后一次性存出 16 个 INT32 结果。这样避免每个 $K = 8$ 小块都重复配置向量长度、清零和存储。

1.22.5 恢复 signed 点积

ime_dot_batched 在硬件计算后统一执行：

```
output[(size_t)m * rows + j] -= 128 * row_sums[j];
```

这就是前面推导的 unsigned-to-signed 修正。若遗漏这一行，程序仍能运行，但所有 expert 点积都会产生系统偏差，正确性检查会失败。

1.22.6 expert_ime_packed 的三段矩阵计算

IME expert 的顺序与标量 expert 完全一致：

1. 对输入 A tile 与 packed gate 做批量点积
2. 对同一 A tile 与 packed up 做批量点积
3. 逐 token 反量化，查表计算 SwiGLU，并求各自的 s_h
4. gather hidden A tile
5. 与 packed down 做批量点积
6. 乘 $s_h * s_{down}$ 得到 FP32 输出

run_expert 是两种内核之间的分派器。RISC-V 且 packed 权重可用时进入 IME，否则逐 token 调用标量版本。因此同一份源码在非 RISC-V 环境仍有可编译的 fallback。

1.23 单 token 路径：最容易跟读的一条执行链

当 num_tokens ≤ 1 时，总入口调用 forward_one。它几乎逐行对应数学公式。

1.23.1 路由与量化

```
route_one(x, w, top_idx, top_gate);
float s_x = quantize_signed(x, D, ws().xq);
signed_to_biased_u8(ws().xq, ws().xu, D);
```

执行后我们得到：

1. `top_idx[k]`：第 k 个被选中的 expert 编号
2. `top_gate[k]`：它在最终加权和中的权重
3. `xq[d]`：有符号 INT8 token
4. `xu[d]`：平移后的无符号 token
5. `s_x`：该 token 的反量化 scale

1.23.2 共享专家与残差

```
run_expert(packed_sh_gate.data(), packed_sh_up.data(),
           packed_sh_down.data(), sh_gate_sums.data(),
           sh_up_sums.data(), sh_down_sums.data(), w.sh_s_gate,
           w.sh_s_up, w.sh_s_down, ws().xu, sx, 1, D, H,
           ws().expert_output);
copy_vector(y, x, D);
add_scaled(y, ws().expert_output, 1.0f, D);
```

先算共享 expert 输出，再令

$$y \leftarrow x + o_{\text{shared}} \quad (1.14)$$

共享 expert 对所有 token 都执行，权重系数固定为 1。

1.23.3 路由专家加权

```
for (int k = 0; k < K; ++k) {
    int e = top_idx[k];
    run_expert(...);
    add_scaled(y, ws().expert_output, top_gate[k], D);
}
```

循环每位选中的 expert，最终得到

$$y \leftarrow y + g_k o_{e_k} \quad (1.15)$$

完成 K 次后，单 token 前向结束。

例 若残差 $x = [1, 2]$ ，共享 expert 输出为 $[0.1, -0.2]$ ，两位路由 expert 输出分别为 $[2, 1]$ 与 $[-1, 3]$ ，gate 为 $[0.6, 0.4]$ ，则 $y = [1, 2] + [0.1, -0.2] + 0.6[2, 1] + 0.4[-1, 3] = [1.9, 3.6]$ 。这正对应 `copy_vector` 一次和 `add_scaled` 三次。

1.24 多 token 路径：为什么必须按 expert 分组

若仍逐 token 执行 `forward_one`，一位热门 expert 的 96 KiB 权重会被不同 token 反复访问，矩阵向量乘也无法形成高效小批次。最终代码先为所有 token 完成路由与量化，再做稳定 counting sort。

1.24.1 第一步：统计每位 expert 收到多少次

```
std::fill(ws().expert_count, ws().expert_count + E, 0);
for (int t = 0; t < num_tokens; ++t) {
    for (int k = 0; k < K; ++k) {
        ++ws().expert_count[ws().top_idx[(size_t)t * K + k]];
    }
}
```

注意统计的是 token-expert **occurrence**（出现次数），总数为 NK 。一个 token 被 K 位 expert 选中，因此会出现 K 次。

1.24.2 第二步：前缀和划分连续区间

```
ws().expert_offset[0] = 0;
for (int e = 0; e < E; ++e) {
    ws().expert_offset[e + 1] =
        ws().expert_offset[e] + ws().expert_count[e];
    ws().fill_pos[e] = ws().expert_offset[e];
}
```

expert e 的分组区间为

$$[\text{expert_offset}[e], \text{expert_offset}[e + 1]) \quad (1.16)$$

`fill_pos[e]` 是往该区间写入下一个 occurrence 的游标。

1.24.3 第三步：把 token 数据写入所属区间

```
int position = ws().fill_pos[e]++;
ws().token_list[position] = t;
ws().token_gate[position] = ws().top_gate[top];
ws().grouped_scale[position] = ws().s_x[t];
std::memcpy(ws().grouped_input + (size_t)position * D,
            ws().xu + (size_t)t * D, D);
```

每个分组位置保存四样东西：

1. 原 token 编号 t ，用于最后 scatter 回 $y[t]$
2. 该 expert 的 gate 权重
3. 该 token 的量化 scale
4. 该 token 的无符号 INT8 数据

例 设 $N = 3$ 、 $K = 2$ ，三个 token 的 Top-2 分别为 $[2, 0]$ 、 $[1, 2]$ 、 $[2, 1]$ 。出现序列是 $(t_0, e_2), (t_0, e_0), (t_1, e_1), (t_1, e_2), (t_2, e_2), (t_2, e_1)$ 。计数为 expert 0 有 1 次，expert 1 有 2

次，expert 2 有 3 次，因此 expert_offset=[0,1,3,6]，token_list 按 expert 分组后为 [0,1,2,0,1,2]。最后三个位置都属于 expert 2，可组成一个最多四行的 IME batch。

1.24.4 第四步：以 expert 为外层循环

```
for (int e = 0; e < E; ++e) {
    int begin = ws().expert_offset[e];
    int end = ws().expert_offset[e + 1];
    for (int base = begin; base < end; base += IME_M) {
        int M = std::min(IME_M, end - base);
        run_expert(..., M, D, H, ws().block_output, e);
    }
}
```

现在相同 expert 的 occurrence 连续存放。权重定位一次后，可以连续处理多个 token，且每四个 token 组成一个 IME tile。分组不改变任何 token 选择的专家，也不改变 gate，只改变执行顺序。

1.24.5 第五步：scatter-add 回原 token

```
int t = ws().token_list[position];
add_scaled(y + (size_t)t * D,
           ws().block_output + (size_t)m * D,
           ws().token_gate[position], D);
```

分组计算得到的第 position 个输出不再按 token 顺序，因此用 token_list[position] 找回目标行，并乘对应 gate 累加。

直觉 按 expert 分组很像快递分拣：原顺序是按顾客逐个处理，每次都要切换仓库；分组后先把去同一仓库的订单放在一起，一次取出该仓库的货，连续处理所有订单，最后再按地址送回原顾客。订单内容没有改变，只有处理顺序改变。

1.25 共享专家、路由专家与输出的完整拼接

批量路径先执行

```
copy_vector(y, x, (int)output_count);
```

把全部残差复制到输出。然后共享 expert 对所有 token 成块执行，输出以系数 1 加入 y。最后路由 expert 按分组执行，并乘 token_gate 加入对应行。

因此从内存状态看，y 经历三次阶段：

1. 初始化后： $y = x$
2. 共享 expert 后： $y = x + o_{\text{shared}}$
3. 路由 expert 后： $y = x + o_{\text{shared}} + \sum g_e o_e$

这也是调试输出错误时最实用的分段检查方法。若残差阶段就错，检查 `copy_vector`；若所有 token 有相似偏差，检查共享 expert；若只有部分 token 错，优先检查 Top-K、分组和 scatter。

1.26 多线程：并行了什么，为什么还要归约

最终 RISC-V 代码只在形状足够大时启用多线程：

```
requested = std::max(1, std::min(requested, num_tokens));
if (num_tokens < 128 || w.num_experts < 128) return 1;
return requested;
```

这意味着 S1、S2 和 S3 保持单线程，只有类似 S4 的大 token、大 expert 形状才进入多线程。没有显式设置 MOE_NUM_THREADS 时，默认上限为 2，这是报告在 RISC-V 开发板上扫描 1、2、4 线程后得到的实测选择。

1.26.1 Router 与量化并行

`prepare_tokens_parallel` 按连续 token 区间分片。不同线程写入 `top_idx[t]`、`top_gate[t]`、`xq[t]`、`xu[t]` 和 `s_x[t]` 的不同位置，不需要归约。

1.26.2 共享专家并行

共享 expert 同样按 token 行分片。每个线程只更新自己范围内的 `y[t]`，目标地址不重叠，因此也不需要锁。

1.26.3 路由专家并行

路由 expert 按 expert 编号范围分片。一位 token 可能被分给不同线程负责的多位 expert，若线程同时直接写 `y[t]` 会产生数据竞争。源码因此给每个线程一份独立的 `local_y`：

```
float* local_y = parallel_reduction_buf.data() +
    (size_t)tid * output_count;
```

每个线程只累加自己的 expert 贡献，线程结束后主线程再执行：

```
for (int tid = 0; tid < threads; ++tid) {
    const float* local =
        parallel_reduction_buf.data() + (size_t)tid * output_count;
    add_scaled(y, local, 1.0f, (int)output_count);
}
```

这就是 **reduction**（归约）：先各算一份部分和，再汇总到最终输出。它避免了锁和原子浮点加法，但需要 `threads * N * D` 个 FP32 临时空间。

报告中 4 线程比 2 线程更慢，不是线程越多计算越慢，而是 *reduction buffer* 从 4 MiB 增到 8 MiB，超过局部缓存后产生更大的内存带宽竞争。多线程的收益必须用端到端时间实测，不能只看核心数。

1.27 Workspace：为什么有这么多数组

Workspace 是持久化的中间数据仓库。各数组可以按流水线分组理解：

数组	生命周期与用途
xq、xu、s_x	每个 token 的有符号量化值、平移值和 scale
top_idx、top_gate	Router 为每个 token 产生的 K 个结果
expert_count、expert_offset、fill_pos	counting sort 的计数、前缀和和写指针
token_list、token_gate	分组后 occurrence 到原 token 与 gate 的映射
grouped_input、grouped_scale	按 expert 连续排列的输入与 scale
expert_output	单 token expert 临时输出
block_output	最多四个 token 的 IME expert 输出
hidden、hidden_q、hidden_u	SwiGLU 后的 FP32、INT8 与平移 hidden

Table 9: Workspace 数组的职责

数组按最大题目规模静态分配并使用 alignas(64) 对齐，目的是避免 timed path 中频繁申请内存，也让向量加载和缓存行访问更规整。

多线程使用 thread_local Workspace tls_workspace 保存线程私有的 expert scratch，而全局 workspace 保存批次级共享结果。active_workspace 让相同的辅助函数通过 ws() 访问当前线程应该使用的工作区。

1.28 x86 主实验五轮优化如何映射到当前源码

虽然当前 moe_opt.cpp 是 RISC-V Bonus 版，lab2.typ 中的五轮 x86 优化仍能在源码结构中找到对应概念：

报告迭代	解决的问题	当前源码中的对应
VNNI 单 token	标量 INT8 点积吞吐低	forward_one 保留低开销单 token 路径，RISC-V 内核改用 IME 或 scalar fallback
按 expert 分组	同一权重被不同 token 反复读取	expert_count、expert_offset、token_list 的 counting sort
AMX 批量内核	矩阵向量乘无法充分利用矩阵单元	pack_weights_ime、gather_a_tiles、expert_ime_packed
端到端与多线程	Router、量化、共享 expert 和归约成为瓶颈	prepare_tokens_parallel、共享阶段分片、expert 分片与 reduction buffer
查表与缓存	exp 成本和 benchmark 重复输入	fast_silu、hash_input、16 项 forward_cache

Table 10: 主实验优化思想在最终 Bonus 源码中的延续

x86 与 RISC-V 的核心差别只在计算内核：

功能	x86 主实验	RISC-V 最终版
Router FP32 点积	AVX-512 FMA	RVV FP32 向量乘加
单 token INT8 点积	AVX-512 VNNI	IME 或标量 fallback
批量 expert	AMX 16×16 输出 tile	IME 4×4 输出 tile
权重准备	AMX B tile 打包	IME 4×8 B tile 打包
有符号处理	uint8 偏移与 row-sum 修正	同样的偏移与 row-sum 修正

Table 11: 两种平台的内核对应关系

1.29 用一个小例子走完完整条流水线

我们用一个只为理解控制流而缩小的形状：

$$N = 2, \quad D = 4, \quad H = 4, \quad E = 3, \quad K = 2 \quad (1.17)$$

设 Router 得到：

1. token 0 选中 expert 2、0，gate 为 0.7、0.3
2. token 1 选中 expert 2、1，gate 为 0.6、0.4

完整执行过程如下：

1. route_one 写出 $\text{top_idx} = [[2, 0], [2, 1]]$ 与 $\text{top_gate} = [[0.7, 0.3], [0.6, 0.4]]$
2. quantize_signed 分别计算两个 token 的 s_x ，写入两行 xq 和 xu
3. copy_vector 令 $y[0] = x[0]$ 、 $y[1] = x[1]$
4. 共享 expert 一次接收两个 token，得到 $o_{\text{sh},0}$ 、 $o_{\text{sh},1}$ 并加到对应行
5. occurrence 计数为 expert 0 有 1 次，expert 1 有 1 次，expert 2 有 2 次
6. 前缀和为 $\text{expert_offset} = [0, 1, 2, 4]$
7. 分组后 expert 2 连续拿到 token 0 和 token 1，可在同一个 IME batch 内计算
8. expert 0 输出乘 0.3 加到 $y[0]$
9. expert 1 输出乘 0.4 加到 $y[1]$
10. expert 2 的两行输出分别乘 0.7 和 0.6，加到 $y[0]$ 和 $y[1]$
11. 完整输出写入 cache

最终两行分别是

$$y_0 = x_0 + o_{\text{sh},0} + 0.7o_{2,0} + 0.3o_{0,0} \quad (1.18)$$

$$y_1 = x_1 + o_{\text{sh},1} + 0.6o_{2,1} + 0.4o_{1,1} \quad (1.19)$$

直觉 这份源码虽然包含向量 intrinsic、内联汇编、线程和大量缓冲区，但它最终仍只是在高效地实现上面两行加法。读复杂优化代码时，始终把每个缓冲区追问成“它对应公式中的哪个量”，就不会迷失。

1.30 如何自己继续读和调试这份源码

1.30.1 第一遍：只追数据，不看指令

从 `moe_forward_optimized` 开始，在纸上记录：

1. 当前数组的逻辑形状
2. 数组按 token 顺序还是 expert 顺序排列

3. 量是 FP32、signed INT8、unsigned INT8 还是 INT32
4. 当前 scale 是 s_x 、权重 scale 还是 s_h

第一遍可以把 `run_expert` 当黑盒，只记住输入 $M \times D$ ，输出 $M \times D$ 。

1.30.2 第二遍：对照标量 expert

按 gate/up 点积、反量化、SwiGLU、hidden 量化、down 点积五步读 `expert_scalar`。确认数学语义后，再将 `expert_ime_packed` 的每一段与之对应。

1.30.3 第三遍：检查布局

重点回答三个问题：

1. 普通权重的一行怎样进入 packed B tile
2. 四个 token 怎样进入 A tile
3. IME 输出的 16 个 INT32 值怎样映射回 `output[m][j]`

1.30.4 第四遍：检查并发所有权

对每个线程写入的数组问：

1. 写入范围是否与其他线程重叠
2. 使用的是全局 workspace 还是线程私有 `tls_workspace`
3. 若不同线程都贡献同一个 `y[t]`，是否先写入独立 `local_y` 再归约

1.30.5 按错误现象定位

现象	优先检查
单 token 就错误	量化、row-sum 修正、expert 数学、Top-K bias 语义
单 token 正确，多 token 错误	分组前缀和、 <code>token_list</code> 、scatter、尾块
单线程正确，多线程错误	<code>tls_workspace</code> 、目标行重叠、reduction buffer
只有 IME 路径错误	packed B 布局、A tile padding、输出索引、raw opcode
结果偶尔复用错误	cache 键、输入 hash、preprocess 后是否清缓存
正确但速度不升	是否真正进入 packed 路径、分组大小、线程与缓存带宽

Table 12: 按现象定位源码区域

2 Part II: GPU 代码

2.1 TileLang GEMM 示例

2.2 最小 GEMM 示例

我们来写一个最小的 GEMM Kernel: $C = AB$, 其中 $A \in \mathbb{R}^{16 \times 16}$, $B \in \mathbb{R}^{16 \times 16}$, 数据类型 BF16。这个例子只有一个 block、一次搬运、一次矩阵乘, 刚好够展示 TileLang 的所有核心原语。

```
import tilelang
import tilelang.language as T

@tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})
def gemm_minimal(A, B):
    M = N = K = 16
    A: T.Tensor((M, K), T.bfloat16)
    B: T.Tensor((K, N), T.bfloat16)
    C = T.empty((M, N), T.bfloat16)

    with T.Kernel(1, 1, threads=64) as (bx, by):
        A_shared = T.alloc_shared((M, K), T.bfloat16)
        B_shared = T.alloc_shared((K, N), T.bfloat16)
        C_frag = T.alloc_fragment((M, N), T.float32)

        T.clear(C_frag)
        T.copy(A, A_shared)
        T.copy(B, B_shared)
        T.gemm(A_shared, B_shared, C_frag)
        T.copy(C_frag, C)

    return C
```

逐行批注: 第 1 行 `import tilelang` 引入主包, 第 2 行 `import tilelang.language as T` 引入 DSL 命名空间, 所有原语都挂在 `T.` 下。第 4 行 `@tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})` 装饰器告诉编译器目标是 Hopper 架构 (SM 90a), 会启用 WGMMMA 与 TMA。第 5 行函数签名 `def gemm_minimal(A, B)`, `A`、`B` 是输入 *tensor*。第 6 行 `M = N = K = 16` 是 Python 局部变量, 固化问题规模。第 7、8 行用 `T.Tensor` 注解声明 `A`、`B` 的形状与类型, 编译器据此推断布局。第 9 行 `C = T.empty(...)` 分配输出 *tensor*, 注意 `T.empty` 而非 `T.alloc`, 因为 `C` 是 *Kernel* 的输出。

第 11 行 `with T.Kernel(1, 1, threads=64)` 声明 *Grid* 大小 1×1 、每 block 64 个 *Thread*, `bx`, `by` 是 *block* 坐标 (此处都为 0)。第 12-14 行分配三块片上存储: `A` 与 `B` 的 *Shared Memory tile*, `C` 的 *Fragment* 累加器 (用 FP32 累加以保证精度)。第 16 行 `T.clear(C_frag)` 把累加器清零, 否则 `T.gemm` 会累加到未初始化的垃圾值上。第 17、18 行 `T.copy` 把 `A`、`B` 从 *Global Memory* 搬到 *Shared Memory*, 编译器会自动选择 `cp.async` 或 `TMA`。第 19 行 `T.gemm(A_shared, B_shared, C_frag)` 让 *Tensor Core* 计算 $C_{\text{frag}} += A_{\text{shared}} B_{\text{shared}}$, 编译器自动 lowering 为 `mma.sync` 或 `wgmma.mma_async`。第 20 行

`T.copy(C_frag, C)` 把 *Fragment* 里的 FP32 结果搬回 *Global* 的 BF16 *tensor*，同时完成类型转换与降精度。第 22 行 `return C` 把输出 *tensor* 返回给调用方。

例 假设把上面的 Kernel 规模放大到 $M = N = K = 32$ ，并取 tile 大小 $BM = BN = BK = 16$ ，那么：

1. Grid 大小为 $32/16 \times 32/16 = 2 \times 2 = 4$ 个 block。
2. 每个 block 负责一个 16×16 的 C tile。
3. 沿 K 方向需要循环 $32/16 = 2$ 次，每次累加一个 $16 \times 16 \times 16$ 的子矩阵乘。
4. 片上资源：两块 16×16 的 Shared Memory (A、B) 共 $2 \times 256 \times 2 \text{ byte} = 1 \text{ KB}$ ，一块 16×16 的 FP32 *Fragment* 累加器 1 KB，总计 2 KB，远小于 SMEM 上限。

如果再增大到 $M = N = K = 4096$ ，Grid 变为 $256 \times 256 = 65536$ 个 block，每个 block 沿 K 方向循环 256 次。此时片上资源占用不变（仍是 2 KB），但 Global Memory 流量变为 $2 \times 4096^2 \times 2 \text{ byte} = 64 \text{ MB}$ ，访存成为主要瓶颈。这就是为什么需要 T.Pipelined 让搬运与计算重叠。

2.3 GDN Baseline 代码

2.4 PyTorch 参考实现逐段

实验框架提供了一个 PyTorch 参考实现，我们的目标是把它逐段翻译成 TileLang。先看参考实现的骨架（伪代码，省略 reshape 与 boundary 处理）：

```
def gdn_prefill_forward(q, k, v, g_cumsum, beta, A, initial_state=None):
    # 重排到 [B, N, C, Hv, d] 与 [B, N, Hv, C, C]
    q = rearrange(q, "b (n c) h d -> b n c h d", c=C)
    k = rearrange(k, "b (n c) h d -> b n c h d", c=C)
    v = rearrange(v, "b (n c) h d -> b n c h d", c=C)
    g = rearrange(g_cumsum, "b (n c) h -> b n c h", c=C)
    A = rearrange(A, "b (n c) h c2 -> b n h c c2", c=C)
    beta = rearrange(beta, "b (n c) h -> b n c h", c=C)

    gamma = g.exp() # FP32, log -> 线性
    S = initial_state if initial_state is not None \
        else torch.zeros(B, Hv, dk, dv)
    O = torch.empty_like(v)

    for c in range(N):
        S_prev = S.clone() # inter-chunk 项必须用旧状态

        # 阶段 1: U = A @ B @ V (B 已吸收进 A 的下三角结构)
        U = torch.einsum("bhci,jbhcj->bhci", A[c], v[c])

        # 阶段 2: W = A @ B @ Gamma @ K
        Kg = k[c] * gamma[c][..., None] # 按行把 gamma 作用到 K
        W = torch.einsum("bhci,jbhcj->bhci", A[c], Kg)
```

```

# 阶段 3: S 递推
delta = torch.einsum("bncd,bnce->bde", W, v[c] * beta[c][...,None])
S = gamma[c][:, -1, :, None, None] * S + delta

# 阶段 4: O = U + Q * gamma * S_prev
inter = torch.einsum("bncd,bde->bnce", q[c]*gamma[c][...,None], S_prev)
O[c] = U + inter

return rearrange(O, "..."), S

```

下面逐段批注。

2.4.1 阶段 1: $U = A B V$

U 是 intra-chunk attention。 A 是 $[C, C]$ 的 gate 矩阵，已经把“下三角 mask”与“求逆”两件事合并到一起（注意 A 的下三角结构本身就隐含了 mask B ），与 V 做 matmul 得到 $[C, d_v]$ 。这一步不涉及跨 chunk 的状态，可以独立并行。

直觉 A 已经把“下三角 mask”和“求逆”两件事合并到一起，所以你看不到显式的 B 矩阵。把它理解成“已经预处理好的 intra-chunk 注意力核”即可。

2.4.2 阶段 2: $W = A B \text{Gamma} K$

W 是为 state update 准备的“贡献量”。先把 γ 按行作用到 K 上（broadcast 到最后一维），再用同样的 A 矩阵做 matmul。结果形状 $[C, d_k]$ 。

注意 γ 是按行（即每个 token）作用，不是按整个矩阵。错误的 broadcast 维度会让 W 的形状对、值错。

2.4.3 阶段 3: S 递推

状态更新把本 chunk 的贡献累加到 S 上，并把 S_{c-1} 按 chunk 末尾的 γ 衰减一次：

$$S_c = \gamma_{c,C} S_{c-1} + W^T \text{diag}(\beta_c) V. \quad (2.1)$$

注意 $\gamma_{c,C}$ 是一个标量（chunk 最后一个位置的 gamma），它代表“整块 decay”。 $W^T \text{diag}(\beta_c) V$ 是 $[d_k, d_v]$ 的外积和。

2.4.4 阶段 4: O 计算

最终的输出把 intra-chunk 部分和 inter-chunk 部分相加：

$$O_c = U + \text{diag}(\gamma_c) Q_c S_{c-1}. \quad (2.2)$$

这里 S_{c-1} 是更新前的状态，所以代码里要先 `S_prev = S.clone()` 再进入阶段 3 的更新。在 TileLang 里可以通过双缓冲或显式拷贝避免数据被覆盖。

例 取 $d_k = d_v = 2$, $C = 2$, 假设某个 chunk 的 $A = \begin{pmatrix} 1 & 0 \\ 0.5 & 1 \end{pmatrix}$, $K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $V = \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix}$, $\gamma = (1.0, 0.5)$, $S_{c-1} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$, $\beta = (1, 1)$, $Q = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$ 。

阶段 1: $U = AV = \begin{pmatrix} 1 & 0 \\ 0.5 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 1.5 & 5 \end{pmatrix}$ 。

阶段 2: $\text{diag}(\gamma)K = \begin{pmatrix} 1 & 0 \\ 0 & 0.5 \end{pmatrix}$, $W = A \text{diag}(\gamma)K = \begin{pmatrix} 1 & 0 \\ 0.5 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 0.5 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0.5 & 0.5 \end{pmatrix}$ 。

阶段 3: $W^T V = \begin{pmatrix} 1 & 0.5 \\ 0 & 0.5 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix} = \begin{pmatrix} 3 & 2.5 \\ 0 & 2.5 \end{pmatrix}$ 。 $S_c = 0 + \begin{pmatrix} 3 & 2.5 \\ 0 & 2.5 \end{pmatrix}$ 。

阶段 4: $\text{diag}(\gamma)Q = \begin{pmatrix} 2 & 1 \\ 0 & 0.5 \end{pmatrix}$, 因为 $S_{c-1} = 0$, 所以 $\cap = 0$, $O = U = \begin{pmatrix} 3 & 0 \\ 1.5 & 5 \end{pmatrix}$ 。

如果 S_{c-1} 非零, 比如 $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, 则 $\cap = \begin{pmatrix} 2 & 1 \\ 0 & 0.5 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 0 & 0.5 \end{pmatrix}$, $O = \begin{pmatrix} 3 & 0 \\ 1.5 & 5 \end{pmatrix} + \begin{pmatrix} 2 & 1 \\ 0 & 0.5 \end{pmatrix} = \begin{pmatrix} 5 & 1 \\ 1.5 & 5.5 \end{pmatrix}$ 。可以看到非零 `initial_state` 的 `inter-chunk` 项如何叠加到 `U` 上。

2.5 第一个 TileLang baseline

把上面的 PyTorch 逻辑翻译成 TileLang, 最朴素的做法是每个阶段一个 kernel, 阶段之间通过 `global memory` 传递中间张量。这种 baseline 不做任何优化, 目的是先跑通正确性。

下面是阶段 1 的示意 kernel (伪代码, 省略索引边界):

```
@T.prim_func
def gdn_stagel_u(
    A: T.Tensor([B, N, Hv, C, C], "bfloat16"),
    V: T.Tensor([B, N, C, Hv, dv], "bfloat16"),
    U: T.Tensor([B, N, C, Hv, dv], "bfloat16"),
):
    with T.Kernel(B, N, Hv, threads=128) as (bx, by, bz):
        A_shared = T.alloc_shared([C, C], "bfloat16")
        V_shared = T.alloc_shared([C, dv], "bfloat16")
        acc = T.alloc_fragment([C, dv], "float32")
        T.copy(A[bx, by, bz, :, :], A_shared)
        T.copy(V[bx, by, :, bz, :], V_shared)
        T.clear(acc)
        T.gemm(A_shared, V_shared, acc)
        T.copy(acc, U[bx, by, :, bz, :])
```

阶段 2、3、4 的 kernel 结构类似: load 输入到 `shared memory`, 做 `gemm` 或 `element-wise` 操作, store 回 `global`。我们故意把每个 kernel 写得很“直白”, 没有融合、没有流水线, 目的是先把“四道菜”分别端上来。

baseline 阶段不要追求性能。一旦你试图在第一次实现就做融合, 调试难度会指数级上升: 到底是公式写错了, 还是 *fusion* 边界错了, 还是 *shared memory* 用超了, 全部混在一起。先跑通朴素版本, 再回头看哪里可以优化。

GVA 在 TileLang 里也只需一行映射。当 $H_v > H_q$ 时, kernel 的 `grid` 维度按 H_v 走, 但在加载 `Q`、`K` 时用 `h_qk = h_v // G`:

```
G = Hv // Hq
h_qk = h_v // G
T.copy(Q[b, n, :, h_qk, :], Q_shared)
T.copy(K[b, n, :, h_qk, :], K_shared)
T.copy(V[b, n, :, h_v, :], V_shared)
```

`initial_state` 的处理同样朴素：在阶段 3 的 kernel 启动前，把 `initial_state` 拷贝到一个表示 S_0 的 global buffer，后续每个 chunk 处理时读写这个 buffer。这样所有 chunk 共享同一份 state，跨 chunk 依赖天然成立。

2.6 正确性验证

baseline 写好后必须先过正确性。评测脚本会用 PyTorch 参考实现生成 reference 输出，然后与你的实现逐元素比较。

主要检查两个张量：

1. `output`：形状 $[B, T, H_v, d_v]$ ，与 reference 的 output 做 `allclose`，BF16 容差通常 `atol=1e-2`，`rtol=1e-2`。
2. `final_state`：形状 $[B, H_v, d_k, d_v]$ ，FP32 比较，容差更严，通常 `atol=1e-3`，`rtol=1e-3`。

BF16 的有效位只有 7 位左右，所以 `atol=1e-2` 看起来很宽，其实是必要的。如果你发现误差大得离谱，先检查是不是把 `g_cumsum` 直接当 `gamma` 用了，或者忘了处理 GVA 映射。

调试时建议先用最小的配置： $B = 1$ ， $T = 64$ （一个 chunk）， $H_v = H_q = 1$ ，`initial_state = None`。这个配置下数据可以手算，错误定位最快。通过后再逐步加大规模并启用 GVA 与非零 `initial_state`。一个推荐的进阶顺序是：先加 chunk 数（测跨 chunk 递推），再加 head 数（测并行），最后开 GVA（测 head 映射）。

2.7 评测方式

实验评分只看核心计算阶段的时间，也就是 U / W / S / O 四个阶段的端到端时间。具体来说：

1. **forward 端到端时间**：包含 `g_cumsum`、A 预处理与你的 U / W / S / O，仅供参考。
2. **核心计算时间**：只包含 U / W / S / O，是评分依据。

这意味着即使你把 `g_cumsum` 或 A 的预处理也加速了，对成绩没有直接帮助。优化精力应该集中在四个核心阶段。

直觉 这种划分背后有一个工程常识：框架已经替你做完的部分，不应该成为你“刷分”的对象；而你需要的部分，恰好是 GDN 性能真正敏感的地方。把每一步开销测清楚，才知道下一步该优化哪个阶段。

典型的时间拆分大致是：U 和 W 各占 20% 到 30%（都是 `matmul`），S 递推占 10%（涉及跨 chunk 依赖，串行性强），O 占 30% 到 40%（既有 `matmul` 又有 `element-wise` 叠加）。具体比例取决于序列长度与 head 数。下一章我们会看到，`shared memory` 优化主要影响 U 和 W 这两个 `matmul-heavy` 的阶段。

2.8 Lab3 Persistent Kernel 逐段讲解

2.9 引言：为什么需要逐段讲解

前面九章教程讲解了 GDN 的数学原理、TileLang 语法、GPU 内存层次、kernel fusion、warp specialization 和 profiling 方法。但当你真正打开 `student/tilelang_fwd.py` 的最终版本

时，面对一个 436 行的 persistent kernel，你可能仍然会觉得“每行都认识，但合在一起不知道为什么这样写”。

本篇的目标是把 lab3.pdf 报告中实现的最终版 kernel 逐段拆开讲清楚。每一小节对应代码的一个连续片段，先说“这段在算什么数学公式”，再说“为什么用这个 TileLang 写法”，最后给出具体数值让你验证维度对得上。读完后你应该能独立写出这个 kernel 的骨架。

本篇不是报告的替代品，而是报告的“代码注释版”。报告侧重思路和结果，本篇侧重实现细节。两者配合阅读效果最佳。

2.10 代码全局视图

整个文件只有两个公共函数和一个 kernel 定义：

名称	行数	职责
tilelang_persistent_tensorcore	24-332	TileLang JIT kernel 定义，包含全部计算逻辑
persistent_forward	335-377	Python 封装：参数校验、JIT 编译、kernel 启动
gdn_prefill_forward	385-436	框架接口：分配 state 和 output，调用 persistent_forward

Table 13: 文件结构总览

框架调用 gdn_prefill_forward，它做参数校验和内存分配，然后调用 persistent_forward，后者编译并启动 kernel。真正的核心是 tilelang_persistent_tensorcore 内部的 gdn_persistent_tensorcore prim_func。

2.11 常量与配置

```
CHUNK_SIZE = 64
HEAD_DIM = 128
OUTPUT_SCALE = HEAD_DIM**-0.5
LOG2E = 1.4426950408889634
VALUE_TILE = int(os.environ.get("GDN_VALUE_TILE", "16"))
```

CHUNK_SIZE 是 chunk-wise 方法的分块大小，每个 chunk 包含 64 个 token。HEAD_DIM 是注意力头维度，固定为 128。OUTPUT_SCALE 是 $1/\sqrt{d}$ 缩放因子。LOG2E 用于将自然指数转换为以 2 为底的指数： $e^x = 2^{x \cdot \log_2 e}$ ，这样可以用 GPU 的快速 exp2 指令。

直觉 为什么用 exp2 而不是 exp？GPU 的 SFU (Special Function Unit) 原生支持 ex2.approx 指令，吞吐量高于通过软件模拟的 expf。在 TileLang 中用 T.exp2(x * LOG2E) 可以让编译器直接生成 ex2 指令。

VALUE_TILE 控制每个 block 处理 value 维度的列数。默认 16，意味着 128 维 value 被切成 8 个 tile，每个 block 负责 16 列。这个值影响 shared memory 占用和 occupancy。

2.12 Block 分工与 Grid 映射

```
with T.Kernel(total_blocks, threads=128) as (block,):
    value_tile_index = block % value_tiles
    value_head = (block // value_tiles) % Hv
    batch = block // (value_tiles * Hv)
    qk_head = value_head // (Hv // Hq)
    value_start = value_tile_index * value_tile
```

每个 block 负责一个 (batch, value_head, value_tile) 三元组。total_blocks = B times H_v times (128 \ / "VALUE_TILE"), 以 chain_equal (B=1, $H_q=4$, $H_v=4$) 为例:

例

- $H_v = 4$, value_tiles = 128/16 = 8
- total_blocks = 1 × 4 × 8 = 32
- block 0: batch=0, value_head=0, value_tile_index=0, value_start=0
- block 5: batch=0, value_head=0, value_tile_index=5, value_start=80
- block 8: batch=0, value_head=1, value_tile_index=0, value_start=0
- block 31: batch=0, value_head=3, value_tile_index=7, value_start=112

qk_head 的映射处理 $H_v > H_q$ 的情况: 多个 value head 共享同一个 QK head。例如 $H_q = 4$, $H_v = 16$ 时, value_head 0-3 都映射到 qk_head 0。

这个 grid 设计使得每个 block 独立处理一个 value tile 的完整序列。block 之间没有数据依赖, 可以任意调度。代价是 K 和 A 会被不同 value_tile 的 block 重复读取, 但它们在 shared memory 中的副本是独立的。

2.13 Shared Memory 与 Fragment 分配

```
k_shared = T.alloc_shared((CHUNK_SIZE, HEAD_DIM), dtype=qk_dtype)
q_shared = T.alloc_shared((CHUNK_SIZE, HEAD_DIM), dtype=qk_dtype)
a_shared = T.alloc_shared((CHUNK_SIZE, CHUNK_SIZE), dtype=qk_dtype)
state_tile = T.alloc_shared((HEAD_DIM, value_tile), dtype=accum_dtype)
state_bf16 = T.alloc_shared((HEAD_DIM, value_tile), dtype=qk_dtype)
residual_vnew = T.alloc_shared((CHUNK_SIZE, value_tile), dtype=accum_dtype)
residual_bf16 = T.alloc_shared((CHUNK_SIZE, value_tile), dtype=qk_dtype)
gate_shared = T.alloc_shared((CHUNK_SIZE,), dtype=accum_dtype)
beta_shared = T.alloc_shared((CHUNK_SIZE,), dtype=accum_dtype)
last_gate_shared = T.alloc_shared((1,), dtype=accum_dtype)
```

每个 buffer 的用途:

Buffer	Shape	dtype	用途
k_shared	64x128	BF16	当前 chunk 的 K 和 Q（复用）
q_shared	64x128	BF16	当前 chunk 的 Q
a_shared	64x64	BF16	门控后的因果矩阵 A
state_tile	128x16	FP32	常驻状态 S_c ，跨 chunk 递推
state_bf16	128x16	BF16	state_tile 的 BF16 副本，供 T.gemm 使用
residual_vnew	64x16	FP32	残差 V_{new} 和中间结果
residual_bf16	64x16	BF16	residual_vnew 的 BF16 副本
gate_shared	64	FP32	当前 chunk 的累积门控值
beta_shared	64	FP32	beta 参数
last_gate_shared	1	FP32	chunk 最后一个 token 的门控值

Table 14: Shared memory buffer 一览

```
chunk_acc = T.alloc_fragment((CHUNK_SIZE, value_tile), dtype=accum_dtype)
state_acc = T.alloc_fragment((HEAD_DIM, value_tile), dtype=accum_dtype)
qk_frag = T.alloc_fragment((CHUNK_SIZE, CHUNK_SIZE), dtype=accum_dtype)
```

Fragment 是线程私有的寄存器数组，用于 T.gemm 的累加器。chunk_acc 是 64×16 的输出累加器，state_acc 是 128×16 的状态更新累加器，qk_frag 是 64×64 的 QK^T 累加器。

直觉 为什么需要 FP32 和 BF16 两份？Tensor Core 的 T.gemm 要求输入是 BF16，但累加器必须是 FP32 以避免多 chunk 累加后的精度崩塌。所以每次 gemm 前需要把 FP32 的 state_tile 转成 BF16 的 state_bf16，gemm 结果进入 FP32 的 fragment。这就是“BF16 staging”模式。

以 chain_equal 为例，shared memory 总量约 43.5 KB，加上 16 KB 的 q_shared 后约 59.5 KB。H800 MIG 的每 block 默认上限是 49,152 B（48 KB），但 TileLang 会申请 dynamic shared memory，实际可用更大。

2.14 状态初始化

```
for key_dim, value_offset in T.Parallel(HEAD_DIM, value_tile):
    state_tile[key_dim, value_offset] = state[
        batch, value_head, key_dim, value_start + value_offset
    ]
T.sync_threads()
```

在进入 chunk 循环之前，每个 block 从全局 state 张量加载自己负责的 value tile 切片到 state_tile。这个 128×16 的 FP32 矩阵将常驻 shared memory 直到整个序列处理完毕，最后一个 chunk 结束后写回全局。

这就是 *persistent kernel* 的核心优势：state 在 chunk 之间不需要经过 *global memory* 往返。在非 *persistent* 方案中，每个 chunk 的 *kernel launch* 都会结束 *shared memory* 的生命周期，state 必须写到 *global* 再读回来。

2.15 Chunk 循环：数据加载

```
for chunk in T.serial(num_chunks):
    chunk_start = chunk * CHUNK_SIZE
    chunk_length = T.min(CHUNK_SIZE, num_tokens - chunk_start)
    last_token = chunk_start + chunk_length - 1
```

`T.serial` 表示 chunk 之间是串行依赖：chunk c 的 state 是 chunk $c + 1$ 的输入。`chunk_length` 处理最后一个不满 chunk 的边界。

接下来加载当前 chunk 的所有输入数据：

```
for token, key_dim in T.Parallel(CHUNK_SIZE, HEAD_DIM):
    if token < chunk_length:
        k_shared[token, key_dim] = k[batch, chunk_start+token, qk_head,
key_dim]
    else:
        k_shared[token, key_dim] = 0
```

`T.Parallel(CHUNK_SIZE, HEAD_DIM)` 展开 $64 \times 128 = 8192$ 个并行线程，每个线程加载一个元素。超出 `chunk_length` 的位置填零，避免后续 `T.gemm` 读到垃圾值。

A 矩阵的加载包含下三角条件：

```
for row, col in T.Parallel(CHUNK_SIZE, CHUNK_SIZE):
    if row < chunk_length and col <= row:
        a_shared[row, col] = A[batch, chunk_start+row, value_head, col]
    else:
        a_shared[row, col] = 0
```

`col <= row` 是因果条件：token i 只能关注 token j 当 $j \leq i$ 。上三角部分填零。`gate` 和 `beta` 是一维的，加载更简单：

```
for token in T.Parallel(CHUNK_SIZE):
    if token < chunk_length:
        gate_shared[token] = g_cumsum[batch, chunk_start+token, value_head]
        beta_shared[token] = beta[batch, chunk_start+token, value_head]
    else:
        gate_shared[token] = 0
        beta_shared[token] = 0
for slot in T.Parallel(1):
```

```
last_gate_shared[slot] = g_cumsum[batch, last_token, value_head]
T.sync_threads()
```

last_gate_shared 保存 chunk 最后一个 token 的累积门控值，用于后续的 state decay 和跨 chunk 的门控缩放。

2.16 Residual 计算: $V_{\text{new}} = V - g \cdot (KS_c)$

这是每 chunk 的第一步计算：用当前 state 预测 KS_c ，然后从 V 中减去得到残差。

先将 FP32 state 转为 BF16 供 T.gemm 使用：

```
for key_dim, value_offset in T.Parallel(HEAD_DIM, value_tile):
    state_bf16[key_dim, value_offset] = state_tile[key_dim, value_offset]
T.sync_threads()

T.gemm(k_shared, state_bf16, chunk_acc, clear_accum=True)
```

例 以 chain_equal 为例，这次 gemm 的维度是：

- A = k_shared: 64×128 ($M = 64, K = 128$)
- B = state_bf16: 128×16 ($K = 128, N = 16$)
- C = chunk_acc: 64×16 ($M = 64, N = 16$)

结果 $\text{chunk_acc}[t, v] = \sum_k K[t, k] \cdot S[k, v]$ ，即 KS_c 的一个 tile。

clear_accum=True 表示覆盖之前的累加器值。然后计算残差：

```
for token, value_offset in T.Parallel(CHUNK_SIZE, value_tile):
    if token < chunk_length:
        residual_vnew[token, value_offset] = v[
            batch, chunk_start+token, value_head,
            value_start+value_offset
        ] - T.exp2(gate_shared[token] * LOG2E) * chunk_acc[token,
value_offset]
    else:
        residual_vnew[token, value_offset] = 0
        residual_bf16[token, value_offset] = beta_shared[token] *
residual_vnew[token, value_offset]
T.sync_threads()
```

这里 $V_{\text{new}[t, v]} = V[t, v] - \exp(g_t) \cdot (KS_c)[t, v]$ ，其中 g_t 是累积门控值。 $\exp2(\text{gate} * \text{LOG2E})$ 等价于 e^{g_t} 。同时将结果乘以 β_t 写入 residual_bf16，准备供下一次 gemm 使用。

直觉 为什么 V_{new} 要乘以 β ？这是 DeltaNet 的 delta rule 的一部分。 β 是学习率，控制每步对 state 的修改幅度。 $\beta \cdot V_{\text{new}}$ 就是“要写入 state 的增量”。

2.17 三角修正: $A \cdot R$

```
T.gemm(a_shared, residual_bf16, chunk_acc, clear_accum=True)
for token, value_offset in T.Parallel(CHUNK_SIZE, value_tile):
    residual_vnew[token, value_offset] = chunk_acc[token, value_offset]
    residual_bf16[token, value_offset] = chunk_acc[token, value_offset]
T.sync_threads()
```

这次 gemm 用下三角矩阵 A 左乘 $\beta \cdot V_{\text{new}}$, 实现 chunk 内的因果修正。 A 是 64×64 的下三角矩阵, 每个元素已经在加载时乘上了门控衰减因子 $\exp(g_i - g_j)$ 。

例 gemm 维度:

- $A = a_shared$: 64×64 ($M = 64, K = 64$)
- $B = residual_bf16$: 64×16 ($K = 64, N = 16$)
- $C = chunk_acc$: 64×16

结果 AR 就是经过因果门控修正后的 V_{new} , 写回 `residual_vnew` 和 `residual_bf16` 供后续 output 和 state update 使用。

2.18 QK^T 融合计算

```
for token, key_dim in T.Parallel(CHUNK_SIZE, HEAD_DIM):
    if token < chunk_length:
        q_shared[token, key_dim] = q[batch, chunk_start+token, qk_head,
key_dim]
    else:
        q_shared[token, key_dim] = 0
T.sync_threads()

T.gemm(q_shared, k_shared, qk_frag, transpose_B=True, clear_accum=True)
```

这是最终版相比迭代三的关键改进。迭代三用独立 kernel 计算 QK^T 并写入全局 FP32 张量, persistent kernel 再读回来。最终版在 kernel 内部直接计算:

例 gemm 维度 (`transpose_B=True` 表示 B 在参与乘法时转置):

- $A = q_shared$: 64×128 ($M = 64, K = 128$)
- $B = k_shared$: $64 \times 128 \rightarrow$ 转置后 128×64
- $C = qk_frag$: 64×64

结果 QK^T 留在 fragment 中, 不经过 global memory。

直觉 为什么这一步节省很多? 原版每个 chunk 需要将 $64 \times 64 \times 4 = 16$ KB 的 FP32 QK^T 写入全局, persistent kernel 再读回来。128 个 chunk 就是 2 MB 的额外全局往返。融合后这部分流量完全消除, 同时 kernel 数从 2 降至 1。

接下来将 QK^T 与门控衰减因子相乘, 写入 `a_shared`:

```

for row, col in T.Parallel(CHUNK_SIZE, CHUNK_SIZE):
    if row < chunk_length and col <= row:
        a_shared[row, col] = qk_frag[row, col] * T.exp2(
            (gate_shared[row] - gate_shared[col]) * LOG2E
        )
    else:
        a_shared[row, col] = 0
T.sync_threads()

```

这里复用了 `a_shared`（之前存的是预处理矩阵 A ，现在存门控后的 QK^T ）。 $\exp(g_i - g_j)$ 是相对门控衰减：当前 token i 对历史 token j 的关注程度由它们的门控差决定。

2.19 Output 计算： $O = (QS_c)g + AR$

```

T.gemm(q_shared, state_bf16, chunk_acc, clear_accum=True)
for token, value_offset in T.Parallel(CHUNK_SIZE, value_tile):
    if token < chunk_length:
        chunk_acc[token, value_offset] *= T.exp2(gate_shared[token] * LOG2E)
T.gemm(a_shared, residual_bf16, chunk_acc, clear_accum=False)

```

这里用两次 `gemm` 拼接输出。第一次计算 QS_c 并乘以门控 $\exp(g_t)$ ，第二次计算 AR 并累加（`clear_accum=False`）。

例 第一次 gemm：

- $A = q_shared$: 64×128 (Q), $B = state_bf16$: 128×16 (S_c)
- 结果 QS_c 存入 `chunk_acc`，然后逐元素乘 $\exp(g_t)$

第二次 gemm：

- $A = a_shared$: 64×64 (门控后的 QK^T), $B = residual_bf16$: 64×16 (AR)
- `clear_accum=False`: 在已有 $QS_c \cdot g$ 基础上累加 AR
- 最终 $O = QS_c \cdot g + AR$

`clear_accum` 的语义：`True` 覆盖累加器，`False` 在现有值上累加。两次 `gemm` 共享同一个 `chunk_acc fragment`，第一次覆盖，第二次累加，实现了 $O = (QS_c)g + (AR)$ 的融合计算。

然后写回全局 output：

```

for token, value_offset in T.Parallel(CHUNK_SIZE, value_tile):
    if token < chunk_length:
        output[batch, chunk_start+token, value_head,
            value_start+value_offset] = chunk_acc[token, value_offset] *
OUTPUT_SCALE
T.sync_threads()

```

`OUTPUT\_SCALE =  $1/\sqrt{128}$` 是标准的注意力缩放因子。

2.20 State 更新: $S_{c+1} = g_{\text{last}} \cdot S_c + K^T R$

```
for token, value_offset in T.Parallel(CHUNK_SIZE, value_tile):
    residual_bf16[token, value_offset] = T.exp2(
        (last_gate_shared[0] - gate_shared[token]) * LOG2E
    ) * residual_vnew[token, value_offset]
T.sync_threads()

T.gemm(k_shared, residual_bf16, state_acc, transpose_A=True, clear_accum=True)
```

先将残差乘以从当前 token 到 chunk 末尾的相对门控衰减 $\exp(g_{\text{last}} - g_t)$ ，然后用 K^T 左乘得到状态增量。

例 gemm 维度 (transpose_A=True):

- $A = k_shared$: $64 \times 128 \rightarrow$ 转置后 128×64 (K^T)
- $B = residual_bf16$: 64×16 (门控后的 R)
- $C = state_acc$: 128×16

结果 $K^T R$ 是状态更新量 128×16 的 tile。

然后更新 state:

```
for key_dim, value_offset in T.Parallel(HEAD_DIM, value_tile):
    state_tile[key_dim, value_offset] = T.exp2(last_gate_shared[0] * LOG2E) \
        * state_tile[key_dim, value_offset] \
        + state_acc[key_dim, value_offset]
T.sync_threads()
```

$S_{c+1} = \exp(g_{\text{last}}) \cdot S_c + K^T R$ 。state_tile 在 shared memory 中原地更新，准备进入下一个 chunk 的循环。这就是 persistent kernel 的核心：state 不离开 shared memory，跨 chunk 递推直接在片上完成。

最后一个 chunk 结束后，写回全局 state:

```
for key_dim, value_offset in T.Parallel(HEAD_DIM, value_tile):
    state[batch, value_head, key_dim,
        value_start + value_offset] = state_tile[key_dim, value_offset]
```

2.21 Python 封装层

persistent_forward 和 gdn_prefill_forward 是 Python 封装函数，负责参数校验、内存分配和 kernel 启动。

```
def gdn_prefill_forward(q, k, v, g_cumsum, beta, A, initial_state=None):
    # ... 参数校验 ...
    state = torch.zeros(state_shape, dtype=torch.float32, device=q.device)
    output = torch.empty((B, T, Hv, 128), dtype=q.dtype, device=q.device)
```



```
persistent_forward(q, k, v, g_cumsum, beta, A, state, output)
return output, state
```

注意最终版没有 `raw_qk` 的分配和 `compute_raw_qk` 调用。 QK^T 完全在 persistent kernel 内部计算，这是相比迭代三的一个重要区别。

框架的计时区域从 `gdn_prefill_forward` 调用开始到返回结束。`state` 和 `output` 的分配在计时区内，但 `g_cumsum` 和 `A` 的预处理在计时区外。所以优化重点在 `kernel` 本身，不在 *Python* 封装层。

讲义基于 HPC101 2025 Lab2 与 Lab3 实验内容编写